

B5 Mixed-Age SSD 的 Degradation-Aware Index 分配

背景

資料中心和邊緣部署環境中，SSD 不會同時更換。硬碟隨容量需求逐批加入、故障時以新硬碟替換、不同年份的採購批次混用。結果，同一個 storage pool 裡，一顆 2024 年買的 SSD (壽命消耗 5%) 和一顆 2020 年的 SSD (壽命消耗 85%) 同時服務相同的 keyspace，形成 mixed-age all-flash array。

問題出在 KV store (如 RocksDB, TiKV, CockroachDB) 的 index 保護機制對所有 SSD 一視同仁。每個 SSTable 的 Bloom filter 都用相同的 bits-per-key, block cache 的 eviction policy 也不區分底層裝置的狀態。但是，一次 index miss 觸發的 verification read 在不同老化程度的 SSD 上，延遲差距非常大：

- 新 SSD 的 4K random read p99 latency 相對穩定
- 老化 SSD 的 p99 latency 會因為 ECC correction 和 read retry 而大幅上升

換句話說，同樣一次 Bloom filter false positive，如果發生在老化的 SSD 上，tail latency 代價遠高於新 SSD。但現行系統在分配 index memory 時忽略這個差異。在固定的 memory budget 下，對老化裝置保護不足 (miss 代價高)，對新裝置保護過度 (miss 代價低)。

這個問題也有 sustainability 的面向。當老化 SSD 的 tail latency 惡化到無法接受時，維運人員通常會直接換掉硬碟。一個值得探索的問題是：如果軟體層能主動補償，用更多 index memory 來避免讀取老化裝置，是否能讓硬碟在可接受的效能下繼續服役更久？

觀察

- **SSD 老化可能導致 read latency 分布惡化，而且據相關研究呈非線性趨勢。**隨著 PE cycle 消耗，ECC correction 和 read retry 頻率增加，p99 latency 上升的幅度可能遠大於 p50。如果這個趨勢成立，老化 SSD 的問題主要表現在 tail latency，不是平均延遲。這需要從現有文獻中找出具體的量化數據來確認。
- **Bloom filter false positive 的代價取決於底層裝置狀態。**傳統的 Bloom filter 分析 (如 Monkey) 假設每次 I/O 的成本是常數。但在 mixed-age array 中，這個假設不成立。同樣的 false positive rate，在老化裝置上的實際代價遠高於新裝置。
- **NVMe SMART 資訊已經提供了裝置老化的即時指標。**percentage_used, media_errors, unsafe_shutdowns 等欄位可以直接透過 nvme smart-log 取得。這些資訊目前被 KV store 的 index 分配邏輯忽略。
- **RocksDB 已有 per-SSTable 的 Bloom filter，也有 block cache priority 機制，但這些都是 device-oblivious 的。**技術上可以做到 per-SSTable 的差異化保護，只是現在沒有人根據裝置狀態來調整。

試著回答下列問題

- 根據現有文獻 (如 SSD-iq), 不同老化程度的 SSD 在 4K random read 的 latency 分布 (p50, p99, p99.9) 差異有多大? 用 FEMU 模擬不同老化程度的 latency profile 時, 應該設定什麼樣的參數才能忠實反映文獻中的數據?
- RocksDB 的 Bloom filter 目前怎麼分配 bits-per-key? 是 per-column-family 還是 global? 如果要做到 per-SSTable 粒度的 bits-per-key, 需要改動哪些 code path?
 - per-column-family(CF): 一個 DB 有許多 CF, 一個 CF 就是一個 LSM-tree, 一個 CF 下有很多 SSTable
 - bits-per-key 是一個靜態參數, 一個 CF 下共用同一個參數, rocksDB 裡沒有自動調整的機制, 但使用者可以去修改他
 - 應該是修改 table/block_based/block_based_table_builder.cc 下 L72 這個 CreateFilterBlockBuilder function, 但如何在此知道該裝置的老化資訊?
 - 整體 code path 詳見 [M08-learning-notes](#)
- 在固定的 total index memory budget 下, 把更多 Bloom filter bits 分配給老化裝置上的 SSTable, 同時減少新裝置上 SSTable 的 bits, 對整體 p99 read latency 有什麼影響? 什麼樣的分配策略最好?
 - 若發生 False Positive, 老化裝置會浪費大量時間, 大幅提高 p99 read latency, 新裝置浪費的時間則會相對少一些, 因此以上操作可以縮短 p99 read latency。
 - 最好的分配策略見下題。
- Monkey (SIGMOD 2017) 的 optimal Bloom filter allocation 是按 LSM level 分配。如果加入 device penalty 這個維度, cost model 要怎麼修改? 這個擴展後的最佳化問題是否仍然可以高效求解?
 - 原先的 cost model 是 FPR 正比於 level 的大小 T , 加入 device penalty 後, 可以推導出 FPR 除了正比於 T , 還要反比於 device penalty。
 - 原本 Monkey 內的公式到這邊無法再推導下去, 不過由於實際應用時 device penalty 會是已知數, 其數學式應該是可求解。
 - 不過 4/17 會議中老師建議將 device penalty 設為離散的等級制
- RocksDB 的 block cache 有 priority 機制 (LOW, HIGH, BOTTOM)。如果根據裝置老化程度來設定 SSTable 的 cache priority, 在 cache 容量有限時, 能減少多少對老化裝置的 I/O?

- 如果老化裝置的資料放在 cache 裡, 就不用去讀 SSD, 相關程式碼應該在 `table/block_based/block_based_table_reader.cc`、`block_based_table_factory.cc`、`block_based_table_builder.cc` 裡
- 現有決定 priority 的機制是根據資料類型, 在創建 table 時就決定的
- 假設: 有一批同類型的資料, 被讀取的頻率都一樣, cache 大小固定, 原本 cache 管理策略對不同老化狀態的裝置一視同仁, 但現在我加入了老化的影響, 越老的裝置的資料越不會被移出 cache
- 若有夠多 cache 可以放下夠多的老化裝置資料, 可以大幅減少 tail latency
- 由於大幅減少 tail latency, 平均延遲通常也會改善, 但中位數會升高
- 這種 device-aware index allocation 和 device-aware data placement (把冷資料放老化裝置) 是互斥的還是互補的?
 - 在 LSM-tree 的一般有讀有寫情境下, 若有冷資料, 應該會是在較深層, 若要把冷資料放在老裝置, 代表是深層使用老裝置
 - 兩者有些互斥, 前者想減少老化裝置的 FPR 因此分配較多的 memory, 後者會減少讀取到老裝置的次數, 因此分配太多 memory 又會造成浪費
 - 或許可以把讀取頻率當成一項新參數, 那 FPR 應該是要反比於讀取頻率

建議閱讀文獻

1. **SSD-iq: Uncovering the Hidden Side of SSD Performance** (Haas et al., PVLDB 2025). 系統性量測了不同 SSD 型號之間的效能差異, 包含 tail latency 特性, 是理解 SSD 效能非均勻性的重要參考。
2. **B-Trees Are Back: Engineering Fast and Pageable Node Layouts** (Muller et al., SIGMOD 2025). 探討在 hybrid storage (memory + flash) 環境下, B-Tree 的 node layout 如何影響 pageable 效能, 對理解 index structure 在不同 storage 層之間的行為很有幫助。
3. **Monkey: Optimal Navigable Key-Value Store** (Dayan et al., SIGMOD 2017). Bloom filter memory 在 LSM-tree 各 level 之間的最佳分配, 是本題的理論基礎。
4. **Endure: A Robust Tuning Paradigm for LSM Trees Under Workload Uncertainty** (Huynh et al., PVLDB 2022). 在 workload 不確定的情況下如何做 robust 的 LSM tuning, 但假設 I/O cost 均勻。

建議探索的 Codebase

- **RocksDB** (<https://github.com/facebook/rocksdb>): 重點看 `table/block_based/` 下 Bloom filter 的建構與查詢流程, `db/compaction/` 下 SSTable 建構時如何決定 filter policy, 以及 `cache/` 下 block cache 的 priority 機制。

- **FEMU** (<https://github.com/MoatLab/FEMU>): 用來模擬不同老化程度 SSD 的 NVMe emulator, 可以注入不同的 read latency profile。
- **ssdiq** (<https://github.com/gabriel-haas/ssdiq>): SSD-iq 論文的 benchmark 工具, 可用來量測 SSD 的 latency 分布特性。

如何開始？

- 閱讀 SSD-iq 和 Monkey 這兩篇論文。SSD-iq 重點理解 SSD 效能差異的來源和量測方法; Monkey 重點理解 Bloom filter bits 在 LSM level 之間的最佳分配公式。
- 在 RocksDB source code 中找到 Bloom filter bits-per-key 的設定點, 追蹤從 Options 到 BlockBasedTableBuilder::CreateFilter 的 code path, 確認目前是不是所有 SSTable 都用相同的 bits-per-key。
- 用 `nvme smart-log /dev/nvmeXn1` 或 `smartctl` 檢查你手邊的 SSD 的 SMART 資訊, 熟悉 `percentage_used`, `data_units_read`, `data_units_written` 等欄位的意義。
- 用 Gemini 或 ChatGPT Deep Research 搜尋近年 SSD aging / wear-out latency degradation 的學術發表, 找出可直接引用的 latency-vs-PE-cycle 模型或實測數據, 作為後續實驗的裝置老化模型基礎。

初步實驗規劃

實驗的核心目標是回答一個問題: 在固定的 index memory budget 下, 根據 SSD 的老化狀態來差異化分配 Bloom filter bits 和 cache priority, 對 read tail latency 有沒有實質影響？

實驗結構

整體實驗可以分成三個階段, 每個階段的產出是下一個階段的輸入:

- **建立裝置老化模型**。請試著先建立 SSD 老化程度和 read latency 分布之間的對應關係。不同老化程度的 SSD, 4K random read 的 p99 latency 差多少? 這個差距是線性的還是加速惡化的? 注意: 這個模型不需要你自己量測, 而是採用現有文獻中已發表的實驗數據與模型(例如 SSD-iq 的量測結果)。請善用 Gemini 或 ChatGPT 的 Deep Research 功能, 搜尋近年關於 SSD aging、wear-out latency degradation 的學術發表, 找出可直接引用的 latency-vs-PE-cycle 模型或實測數據。
- **分配策略實作與驗證**。在 RocksDB 上實作差異化的 index memory 分配, 然後用 YCSB 等標準 workload 來比較: 同樣的 memory budget, 差異化分配 vs. 均勻分配, p99 read latency 差多少?
- **進階情境探索**。在基本效果確認後, 可以進一步問: 當裝置持續老化時, 分配策略能不能自動追蹤? 差異化分配和 data placement(把冷資料放老化裝置)是互補還是互斥?

軟體元件與角色

你會用到幾個主要的軟體工具, 各自的角色不同:

- **RocksDB** 是實驗的主體。請試著理解它的 Bloom filter 建構流程(從 Options 到 BlockBasedTableBuilder)和 block cache 的 priority 機制。所有的分配策略修改都發生在這裡。入口點在 table/block_based/ 和 cache/ 目錄下。
- **FEMU** 是 NVMe SSD emulator, 讓你可以在沒有實體老化 SSD 的情況下模擬不同的 read latency profile。它的價值在於可控性: 你可以精確設定想要的 latency 分布, 不需要等 SSD 真的老化。
- **fio** 用來做裝置層級的 latency 量測。它產出的 latency histogram 是你校準 FEMU 模型和驗證假設的基準。
- **YCSB** 或 **db_bench**(RocksDB 內建的 benchmark)用來產生 KV workload。你需要能控制 read/write 比例、key distribution、dataset size。

第一步該做什麼

在寫任何程式之前, 先確認事實基礎:

- 從文獻中找出 SSD 老化程度與 read latency 分布的對應關係(例如 SSD-iq 的量測數據)。用 Gemini 或 ChatGPT Deep Research 搜尋近年相關學術發表, 整理出可引用的 latency-vs-PE-cycle 模型。
- 根據文獻中的模型, 在 FEMU 上設定兩個 emulated NVMe device, 一個模擬新 SSD 的 latency profile, 另一個模擬老化 SSD。用 fio 確認兩者的 latency 分布符合文獻中的預期。
- 跑一次 RocksDB 的 db_bench, 用預設的 Bloom filter 設定, 觀察 read path 的 latency 分布。這是你後續所有比較的 baseline。

量測什麼

每組實驗至少需要記錄以下指標:

- **Read latency 分布**: p50, p99, p99.9。p99 是主要比較指標, 因為老化 SSD 的問題主要表現在 tail latency。
- **Throughput**: ops/s。確認 latency 改善不是以犧牲 throughput 換來的。
- **Read amplification**: 每次 user read 觸發多少次 device read。這反映 Bloom filter 和 cache 的攔截效果。
- **Memory 使用量**: 確認所有比較組的 total index memory 一致, 否則比較沒有意義。